TASK 11: DATABASE AUTOMATION USING VIEWS AND PROCEDURES

STUDENT DETAILS

Name: Divya. V
Roll No: E25AI111
Department: B.Sc Computer Science With AI

SCENARIO

The Artisan Crumb Bakery & Pastry Management System frequently generates reports and performs repetitive database operations such as bakery order processing, sales calculation, inventory monitoring, and customer analysis. To improve efficiency and reduce repeated SQL coding, the system automates database operations using Views, Stored Procedures, and Functions. Database automation features are implemented in the existing Bakery Database System.

1. Create Views for Frequently Used Reports

Concept

A View is a virtual table created from one or more SQL queries. It stores the query definition and provides simplified access to frequently used information.

1. Customer Order Report View

Display:

- Customer Name
- Order ID
- Order Date
- Total Amount
- Order Status

SQL Query

```
SQL> CREATE OR REPLACE VIEW Customer_Order_Report AS
  2  SELECT
  3      C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  4      O.Order_ID,
  5      O.Order_Date,
  6      O.Total_Amount,
  7      O.Order_Status
  8  FROM Customer C
  9  JOIN Orders O
 10      ON C.Customer_ID = O.Customer_ID;

View created.
```

Test View

```
SQL> SELECT * FROM Customer_Order_Report;

CUSTOMER_NAME                          ORDER_ID            ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
-------------------------------------- ------------------- --------- ------------ -------------
Bob White                              ORD-505             24-MAR-26         1250 Completed
Jane Smith                             ORD-506             25-MAR-26          720 Completed
Alice Brown                            ORD-507             26-MAR-26         1450 Baking
John Doe                               ORD-508             27-MAR-26          890 Pending
Charlie Green                          ORD-509             28-MAR-26          950 Completed
John Doe                               ORD-510             20-MAR-26         1500 Completed
John Doe                               ORD-511             21-MAR-26         1200 Completed
David Miller                           O005                04-NOV-26          500

8 rows selected.
```

2. Product Sales Report View

Display:

- Bakery Item Name
- Quantity Sold
- Total Revenue

SQL Query

```
SQL> CREATE OR REPLACE VIEW Product_Sales_Report AS
  2  SELECT
  3      P.Product_ID,
  4      P.Title AS Bakery_Item_Name,
  5      SUM(OD.Quantity) AS Quantity_Sold,
  6      SUM(OD.Quantity * OD.Price) AS Total_Revenue
  7  FROM Product P
  8  JOIN Order_Details OD
  9      ON P.Product_ID = OD.Product_ID
 10  GROUP BY P.Product_ID, P.Title;

View created.
```

Test View

```
SQL> SELECT * FROM Product_Sales_Report;

PRODUCT BAKERY_ITEM_NAME          QUANTITY_SOLD TOTAL_REVENUE
------- ------------------------- ------------- -------------
P001    Chocolate Fudge Cake                  3          2250
P002    Sourdough Bread                       2           600
P003    Red Velvet Pastry                     3          1350
P005    Black Forest Cake                     2          1200
```

3. Payment Status View

Display:

- Order ID
- Payment Mode
- Payment Status
- Transaction Amount

SQL Query

```
SQL> CREATE OR REPLACE VIEW Payment_Status_View AS
  2  SELECT
  3       Order_ID,
  4       Payment_Mode,
  5       Payment_Status,
  6       Transaction_Amount
  7  FROM Payment;

View created.
```

Test View

```
SQL> SELECT * FROM Payment_Status_View;

ORDER_ID            PAYMENT_MODE                                       PAYMENT_STATUS                                     TRANSACTION_AMOUNT
------------------- -------------------------------------------------- -------------------------------------------------- ------------------
ORD-507             Debit Card                                         Failed                                                           1450
ORD-507             Credit Card                                        Completed                                                        1450
ORD-508             Debit Card                                         Completed                                                         890
ORD-501             Credit Card                                        Completed                                                         660
ORD-502             UPI                                                Completed                                                         850
ORD-503             Debit Card                                         Completed                                                         360
ORD-504             Net Banking                                        Completed                                                         510
ORD-505             UPI                                                Completed                                                        1250
ORD-506             COD                                                Completed                                                         720

9 rows selected.
```

2. Develop Stored Procedures for Order Processing

Concept

A Stored Procedure is a collection of SQL statements stored in the database that can be executed whenever required. It helps automate repeated operations.

1. Add New Customer Order

Process:

- Insert order details
- Add purchased products
- Calculate total amount

SQL Query

```
SQL> CREATE OR REPLACE PROCEDURE Add_New_Order
  2  (
  3      p_order_id     IN VARCHAR2,
  4      p_customer_id  IN VARCHAR2,
  5      p_order_date   IN DATE,
  6      p_total_amount IN NUMBER
  7  )
  8  AS
  9  BEGIN
 10      INSERT INTO Orders
 11      (
 12          Order_ID,
 13          Customer_ID,
 14          Order_Date,
 15          Total_Amount,
 16          Order_Status
 17      )
 18      VALUES
 19      (
 20          p_order_id,
 21          p_customer_id,
 22          p_order_date,
 23          p_total_amount,
 24          'Pending'
 25      );
 26
 27      COMMIT;
 28  END;
 29  /

Procedure created.
```

Execute Procedure

```
SQL> EXEC Add_New_Order('ORD500', 'C001', SYSDATE, 2500);

PL/SQL procedure successfully completed.
```

2. Update Order Status

Example:

Pending → Shipped → Delivered

SQL Query

```
SQL> CREATE OR REPLACE PROCEDURE Update_Order_Status
  2  (
  3  p_order_id IN VARCHAR2,
  4  p_status IN VARCHAR2
  5  )
  6  AS
  7  BEGIN
  8  UPDATE Orders
  9  SET Order_Status = p_status
 10  WHERE Order_ID = p_order_id;
 11
 12  COMMIT;
 13  END;
 14  /

Procedure created.
```

Execute Procedure

```
SQL> EXEC Update_Order_Status('ORD500', 'Delivered');

PL/SQL procedure successfully completed.
```

3. Generate Customer Purchase History

Input:

- Customer_ID

Output:

- Order Details
- Purchase Amount
- Order Status

SQL Query

```
SQL> CREATE OR REPLACE PROCEDURE Get_Customer_Orders
  2  (
  3      p_customer_id IN VARCHAR2
  4  )
  5  AS
  6  BEGIN
  7      FOR rec IN
  8      (
  9          SELECT Order_ID,
 10                 Order_Date,
 11                 Total_Amount,
 12                 Order_Status
 13          FROM Orders
 14          WHERE Customer_ID = p_customer_id
 15      )
 16      LOOP
 17          DBMS_OUTPUT.PUT_LINE
 18          (
 19              rec.Order_ID || ' ' ||
 20              rec.Total_Amount || ' ' ||
 21              rec.Order_Status
 22          );
 23      END LOOP;
 24  END;
 25  /

Procedure created.
```

Execute Procedure

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> EXEC Get_Customer_Orders('C001');
ORD-508 890 Pending
ORD500 2500 Delivered
ORD-510 1500 Completed
ORD-511 1200 Completed

PL/SQL procedure successfully completed.
```

3. Create Functions for Sales Calculations

Concept

A Function performs a specific calculation and returns a value.

1. Calculate Total Sales

Input:

- Date Range

Output:

- Total Revenue Generated

SQL Query

```
SQL> CREATE OR REPLACE FUNCTION Calculate_Total_Sales
  2  (
  3      p_start_date DATE,
  4      p_end_date   DATE
  5  )
  6  RETURN NUMBER
  7  IS
  8      v_sales NUMBER;
  9  BEGIN
 10      SELECT NVL(SUM(Total_Amount), 0)
 11      INTO v_sales
 12      FROM Orders
 13      WHERE Order_Date BETWEEN p_start_date AND p_end_date;
 14
 15      RETURN v_sales;
 16  END;
 17  /

Function created.
```

Test Function

```
SQL> SELECT Calculate_Total_Sales
  2  (
  3      TO_DATE('01-01-2026', 'DD-MM-YYYY'),
  4      TO_DATE('31-12-2026', 'DD-MM-YYYY')
  5  ) AS Total_Sales
  6  FROM Dual;

TOTAL_SALES
-----------
      10960
```

2. Calculate Customer Total Spending

Input:

- Customer_ID

Output:

- Total Amount Spent

SQL Query

```
SQL> CREATE OR REPLACE FUNCTION Customer_Total_Spending
  2  (
  3      p_customer_id VARCHAR2
  4  )
  5  RETURN NUMBER
  6  IS
  7      v_amount NUMBER;
  8  BEGIN
  9      SELECT NVL(SUM(Total_Amount), 0)
 10      INTO v_amount
 11      FROM Orders
 12      WHERE Customer_ID = p_customer_id;
 13
 14      RETURN v_amount;
 15  END;
 16  /

Function created.
```

Test Function

```
SQL> SELECT Customer_Total_Spending('C001') AS Total_Spending
  2  FROM Dual;

TOTAL_SPENDING
--------------
          6090
```

3. Calculate Product Revenue

Input:

- Product_ID

Output:

- Total Sales Amount

SQL Query

```
SQL> CREATE OR REPLACE FUNCTION Product_Revenue
  2  (
  3      p_product_id VARCHAR2
  4  )
  5  RETURN NUMBER
  6  IS
  7      v_revenue NUMBER;
  8  BEGIN
  9      SELECT NVL(SUM(OD.Quantity * OD.Price), 0)
 10      INTO v_revenue
 11      FROM Order_Details OD
 12      WHERE OD.Product_ID = p_product_id;
 13
 14      RETURN v_revenue;
 15  END;
 16  /

Function created.
```

Test Function

```
SQL> SELECT Product_Revenue('P001') AS Product_Revenue
  2  FROM Dual;

PRODUCT_REVENUE
---------------
           2250
```

4. Automate Repetitive Database Operations

Order Processing Automation

When a customer places an order:

1. Insert Order Details

```
SQL> INSERT INTO Orders
  2  VALUES ('ORD501', 'C002', SYSDATE, 1500, 'Pending');

1 row created.
```

2. Reduce Product Stock

```
SQL> UPDATE Inventory
  2  SET Stock_Quantity = Stock_Quantity - 1
  3  WHERE Product_ID = 'P001';

1 row updated.
```

3. Create Payment Record

```
SQL> INSERT INTO Payment
  2  VALUES ('PAY501', 'ORD501', SYSDATE, 'UPI', 'Completed', 1500);

1 row created.
```

4. Update Order Status

```
SQL> UPDATE Orders
  2  SET Order_Status = 'Delivered'
  3  WHERE Order_ID = 'ORD501';

1 row updated.
```

Inventory Update Automation

Increase Stock After New Supply

```
SQL> UPDATE Inventory
  2  SET Stock_Quantity = Stock_Quantity + 50
  3  WHERE Product_ID = 'P001';

1 row updated.
```

Reduce Stock After Purchase

```
SQL> UPDATE Inventory
  2  SET Stock_Quantity = Stock_Quantity - 2
  3  WHERE Product_ID = 'P001';

1 row updated.
```

Identify Low-Stock Products

```
SQL> SELECT Product_ID, Stock_Quantity, Stock_Status
  2  FROM Inventory
  3  WHERE Stock_Quantity < 10;

PRODUCT STOCK_QUANTITY STOCK_STATUS
------- -------------- ---------------
P005                 0 Out of Stock
P003                 5 Low Stock
```

Report Generation Automation

Daily Sales Report

```
SQL> SELECT Order_Date, SUM(Total_Amount) AS Total_Sales
  2  FROM Orders
  3  GROUP BY Order_Date;

ORDER_DAT TOTAL_SALES
--------- -----------
20-MAR-26        1500
21-MAR-26        1200
09-NOV-26        1500
28-MAR-26         950
24-MAR-26        1250
09-NOV-26        2500
25-MAR-26         720
27-MAR-26         890
26-MAR-26        1450
04-NOV-26         500

10 rows selected.
```

Monthly Revenue Report

```
SQL> SELECT TO_CHAR(Order_Date, 'MON-YYYY') AS Month, SUM(Total_Amount) AS Revenue
  2  FROM Orders
  3  GROUP BY TO_CHAR(Order_Date, 'MON-YYYY');

MONTH                REVENUE
---------------- ----------
MAR-2026               7960
NOV-2026               4500
```

Customer Purchase Report

```
SQL> SELECT Customer_ID, COUNT(Order_ID) AS Orders_Placed, SUM(Total_Amount) AS Total_Spent
  2  FROM Orders
  3  GROUP BY Customer_ID;

CUSTOMER_ID          ORDERS_PLACED TOTAL_SPENT
-------------------- ------------- -----------
C004                             1        1250
C006                             1         500
C003                             1        1450
C005                             1         950
C002                             2        2220
C001                             4        6090

6 rows selected.
```

5. Test Database Objects

Views

Check:

- Correct Data Retrieval
- Report Accuracy

```
SQL> SELECT * FROM Customer_Order_Report;

CUSTOMER_NAME                          ORDER_ID            ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
-------------------------------------- ------------------- --------- ------------ --------------
Bob White                              ORD-505             24-MAR-26         1250 Completed
Jane Smith                             ORD-506             25-MAR-26          720 Completed
Alice Brown                            ORD-507             26-MAR-26         1450 Baking
John Doe                               ORD-508             27-MAR-26          890 Pending
Charlie Green                          ORD-509             28-MAR-26          950 Completed
John Doe                               ORD500              09-NOV-26         2500 Delivered
Jane Smith                             ORD501              09-NOV-26         1500 Delivered
John Doe                               ORD-510             20-MAR-26         1500 Completed
John Doe                               ORD-511             21-MAR-26         1200 Completed
David Miller                           O005                04-NOV-26          500

10 rows selected.
```

```
SQL> SELECT * FROM Product_Sales_Report;

PRODUCT BAKERY_ITEM_NAME          QUANTITY_SOLD TOTAL_REVENUE
------- ------------------------- ------------- -------------
P001    Chocolate Fudge Cake                  3          2250
P002    Sourdough Bread                       2           600
P003    Red Velvet Pastry                     3          1350
P005    Black Forest Cake                     2          1200
```

```
SQL> SELECT * FROM Payment_Status_View;

ORDER_ID             PAYMENT_MODE
-------------------- --------------------------------------------------
PAYMENT_STATUS                                     TRANSACTION_AMOUNT
-------------------------------------------------- ------------------
ORD-507              Debit Card
Failed                                                           1450

ORD-507              Credit Card
Completed                                                        1450

ORD-508              Debit Card
Completed                                                         890

ORD-501              Credit Card
Completed                                                         660

ORD-502              UPI
Completed                                                         850

ORD-503              Debit Card
Completed                                                         360

ORD-504              Net Banking
Completed                                                         510

ORD-505              UPI
Completed                                                        1250

ORD-506              COD
Completed                                                         720

ORD501               UPI
Completed                                                        1500


10 rows selected.
```

Procedures

Test:

- Procedure Execution
- Input Parameters
- Output Results

```
SQL> EXEC Update_Order_Status('ORD500','Delivered');

PL/SQL procedure successfully completed.
```

```
SQL> EXEC Get_Customer_Orders('C001');
ORD-508 890 Pending
ORD500 2500 Delivered
ORD-510 1500 Completed
ORD-511 1200 Completed

PL/SQL procedure successfully completed.
```

Functions

Verify:

- Calculation Accuracy
- Return Values

```
SQL> SELECT Customer_Total_Spending('C001')
  2  FROM Dual;

CUSTOMER_TOTAL_SPENDING('C001')
-------------------------------
                           6090
```

```
SQL> SELECT Product_Revenue('P001')
  2  FROM Dual;

PRODUCT_REVENUE('P001')
-----------------------
                   2250
```

```
SQL> SELECT Calculate_Total_Sales
  2  (
  3  TO_DATE('01-01-2026','DD-MM-YYYY'),
  4  TO_DATE('31-12-2026','DD-MM-YYYY')
  5  )
  6  FROM Dual;

CALCULATE_TOTAL_SALES(TO_DATE('01-01-2026','DD-MM-YYYY'),TO_DATE('31-12-2026','DD-MM-YYYY'))
---------------------------------------------------------------------------------------------
                                                                                        12460
```